

M10 Teaching Book

Docker + Airflow - beginner-first, evidence-first

Teaching standard

Explain -> mental model -> working example -> guided rebuild -> independent attempt -> validate -> recover -> changed transfer -> explain back -> mastery. External visuals support understanding; the book and course-owned labs define the exact competency.

Current baseline

Docker current documentation + Apache Airflow 3.3.1 stable / airflow.sdk authoring. Do not copy older Airflow internal-import patterns when current public Task SDK equivalents exist.

DE10-01 - Why containers exist

OUTCOME

Explain what a container isolates and what it does not.

EXPLAIN

An image is packaged filesystem/config metadata; a container is a running instance that still uses the host kernel. Containers improve reproducibility, not magical security or persistence.

WHY IT MATTERS

A pipeline that works once is not enough. The operational contract must still be reproducible, observable and safe when configuration, time, failures or retries change.

ASSIGNED VISUAL / REFERENCE

[DataTalksClub Zoomcamp - Docker & PostgreSQL DE workshop](#)

Docker/container mental model and hands-on chapters.

[Official: Docker overview](#)

WORKING MODEL / DEMONSTRATION

Compare running `worker.py` on the host with the declared Docker environment.

READ IT LIKE AN ENGINEER

Name the boundary, expected state transition, evidence produced and one failure mode before you run or modify anything.

DE10-01 - Practice, validation and mastery

GUIDED REBUILD

Compare running worker.py on the host with the declared Docker environment. Reproduce the matching course asset once and save concrete evidence.

CHECKPOINT / VALIDATE

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence.

RECOVER

If you call a container a VM, restate kernel sharing and lifecycle.

INDEPENDENT ATTEMPT

Close this page and the external visual. Rebuild the competency from the provided requirement, not by copying the example. Save first attempt even when wrong.

FRESH TRANSFER

Change at least one identifier/configuration/data interval/failure condition and prove the same principle still holds.

EXPLAIN BACK

In 60-90 seconds explain what you built, why the control exists, how you validated it, and what failure would invalidate your claim.

MASTER

Master only when you can reproduce the changed task without the lesson/review open and defend one trade-off.

DE10-02 - Images and containers

OUTCOME

Distinguish immutable image layers from mutable container runtime state.

EXPLAIN

Build once, run many. Container changes disappear when the container is replaced unless persisted externally.

WHY IT MATTERS

A pipeline that works once is not enough. The operational contract must still be reproducible, observable and safe when configuration, time, failures or retries change.

ASSIGNED VISUAL / REFERENCE

[DataTalksClub Zoomcamp - Docker & PostgreSQL DE workshop](#)

Images vs containers.

[Official: Docker run containers](#)

WORKING MODEL / DEMONSTRATION

Trace FROM/COPY/RUN into image, then CMD into a container process.

READ IT LIKE AN ENGINEER

Name the boundary, expected state transition, evidence produced and one failure mode before you run or modify anything.

DE10-02 - Practice, validation and mastery

GUIDED REBUILD

Trace FROM/COPY/RUN into image, then CMD into a container process. Reproduce the matching course asset once and save concrete evidence.

CHECKPOINT / VALIDATE

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence.

RECOVER

If you rely on changes made interactively inside a container, rebuild the image instead.

INDEPENDENT ATTEMPT

Close this page and the external visual. Rebuild the competency from the provided requirement, not by copying the example. Save first attempt even when wrong.

FRESH TRANSFER

Change at least one identifier/configuration/data interval/failure condition and prove the same principle still holds.

EXPLAIN BACK

In 60-90 seconds explain what you built, why the control exists, how you validated it, and what failure would invalidate your claim.

MASTER

Master only when you can reproduce the changed task without the lesson/review open and defend one trade-off.

DE10-03 - Writing a Dockerfile

OUTCOME

Write a small, deterministic Dockerfile with a clear base, workdir, copy/install order and non-root runtime.

EXPLAIN

Cache-friendly build order copies dependency metadata before changing app code. Pin deliberately when reproducibility matters. Avoid secrets in build context/layers.

WHY IT MATTERS

A pipeline that works once is not enough. The operational contract must still be reproducible, observable and safe when configuration, time, failures or retries change.

ASSIGNED VISUAL / REFERENCE

[DataTalksClub Zoomcamp - Docker & PostgreSQL DE workshop](#)

Dockerfile/build chapter.

[Official: Dockerfile reference](#)

WORKING MODEL / DEMONSTRATION

Inspect the provided Python 3.13-slim Dockerfile and .dockerignore.

READ IT LIKE AN ENGINEER

Name the boundary, expected state transition, evidence produced and one failure mode before you run or modify anything.

DE10-03 - Practice, validation and mastery

GUIDED REBUILD

Inspect the provided Python 3.13-slim Dockerfile and .dockerignore. Reproduce the matching course asset once and save concrete evidence.

CHECKPOINT / VALIDATE

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence.

RECOVER

If builds leak local files, inspect build context and .dockerignore.

INDEPENDENT ATTEMPT

Close this page and the external visual. Rebuild the competency from the provided requirement, not by copying the example. Save first attempt even when wrong.

FRESH TRANSFER

Change at least one identifier/configuration/data interval/failure condition and prove the same principle still holds.

EXPLAIN BACK

In 60-90 seconds explain what you built, why the control exists, how you validated it, and what failure would invalidate your claim.

MASTER

Master only when you can reproduce the changed task without the lesson/review open and defend one trade-off.

DE10-04 - Ports

OUTCOME

Explain container ports, EXPOSE, host publishing and loopback binding.

EXPLAIN

EXPOSE documents intended container ports; it does not publish them. Host publication maps host:container and binding to 127.0.0.1 limits local exposure.

WHY IT MATTERS

A pipeline that works once is not enough. The operational contract must still be reproducible, observable and safe when configuration, time, failures or retries change.

ASSIGNED VISUAL / REFERENCE

[DataTalksClub Zoomcamp - Docker & PostgreSQL DE workshop](#)

Ports/networking chapter.

[Official: Dockerfile EXPOSE + publish behavior](#)

WORKING MODEL / DEMONSTRATION

Read 127.0.0.1:8080:8080 in compose.yaml.

READ IT LIKE AN ENGINEER

Name the boundary, expected state transition, evidence produced and one failure mode before you run or modify anything.

DE10-04 - Practice, validation and mastery

GUIDED REBUILD

Read 127.0.0.1:8080:8080 in compose.yaml. Reproduce the matching course asset once and save concrete evidence.

CHECKPOINT / VALIDATE

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence.

RECOVER

If localhost cannot connect, separate app listen address, container port, published host port and firewall.

INDEPENDENT ATTEMPT

Close this page and the external visual. Rebuild the competency from the provided requirement, not by copying the example. Save first attempt even when wrong.

FRESH TRANSFER

Change at least one identifier/configuration/data interval/failure condition and prove the same principle still holds.

EXPLAIN BACK

In 60-90 seconds explain what you built, why the control exists, how you validated it, and what failure would invalidate your claim.

MASTER

Master only when you can reproduce the changed task without the lesson/review open and defend one trade-off.

DE10-05 - Volumes and persistence

OUTCOME

Choose a named volume when data must outlive container replacement.

EXPLAIN

Container writable layers are ephemeral. Volumes are engine-managed persistent stores; bind mounts expose host paths and fit different use cases.

WHY IT MATTERS

A pipeline that works once is not enough. The operational contract must still be reproducible, observable and safe when configuration, time, failures or retries change.

ASSIGNED VISUAL / REFERENCE

[DataTalksClub Zoomcamp - Docker & PostgreSQL DE workshop](#)

Volumes/persistence chapter.

[Official: Docker storage volumes](#)

WORKING MODEL / DEMONSTRATION

Trace pgdata to `/var/lib/postgresql/data`.

READ IT LIKE AN ENGINEER

Name the boundary, expected state transition, evidence produced and one failure mode before you run or modify anything.

DE10-05 - Practice, validation and mastery

GUIDED REBUILD

Trace pgdata to /var/lib/postgresql/data. Reproduce the matching course asset once and save concrete evidence.

CHECKPOINT / VALIDATE

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence.

RECOVER

If data vanishes after recreation, prove where it was written.

INDEPENDENT ATTEMPT

Close this page and the external visual. Rebuild the competency from the provided requirement, not by copying the example. Save first attempt even when wrong.

FRESH TRANSFER

Change at least one identifier/configuration/data interval/failure condition and prove the same principle still holds.

EXPLAIN BACK

In 60-90 seconds explain what you built, why the control exists, how you validated it, and what failure would invalidate your claim.

MASTER

Master only when you can reproduce the changed task without the lesson/review open and defend one trade-off.

DE10-06 - Networks and service communication

OUTCOME

Use service-name DNS for container-to-container communication.

EXPLAIN

Compose creates a project network by default; services on it resolve each other by service name. Do not hardcode changing container IPs.

WHY IT MATTERS

A pipeline that works once is not enough. The operational contract must still be reproducible, observable and safe when configuration, time, failures or retries change.

ASSIGNED VISUAL / REFERENCE

[DataTalksClub Zoomcamp - Docker & PostgreSQL DE workshop](#)

Network/service communication chapter.

[Official: Compose networking](#)

WORKING MODEL / DEMONSTRATION

The api uses DB_HOST=db and both join backend.

READ IT LIKE AN ENGINEER

Name the boundary, expected state transition, evidence produced and one failure mode before you run or modify anything.

DE10-06 - Practice, validation and mastery

GUIDED REBUILD

The api uses DB_HOST=db and both join backend. Reproduce the matching course asset once and save concrete evidence.

CHECKPOINT / VALIDATE

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence.

RECOVER

If service A cannot reach B, inspect network membership, service name and listening port before host port mappings.

INDEPENDENT ATTEMPT

Close this page and the external visual. Rebuild the competency from the provided requirement, not by copying the example. Save first attempt even when wrong.

FRESH TRANSFER

Change at least one identifier/configuration/data interval/failure condition and prove the same principle still holds.

EXPLAIN BACK

In 60-90 seconds explain what you built, why the control exists, how you validated it, and what failure would invalidate your claim.

MASTER

Master only when you can reproduce the changed task without the lesson/review open and defend one trade-off.

DE10-07 - Compose, configuration and secrets

OUTCOME

Define a multi-service stack while separating normal configuration from secrets.

EXPLAIN

Compose manages services, networks and volumes. Sensitive values should not be committed in source; use secrets or an appropriate secret backend.

WHY IT MATTERS

A pipeline that works once is not enough. The operational contract must still be reproducible, observable and safe when configuration, time, failures or retries change.

ASSIGNED VISUAL / REFERENCE

[DataTalksClub Zoomcamp - Docker & PostgreSQL DE workshop](#)

Compose/configuration chapter; course adds current secret handling.

[Official: Docker Compose](#)

[Official: Compose secrets](#)

WORKING MODEL / DEMONSTRATION

Inspect POSTGRES_PASSWORD_FILE and per-service secret grant.

READ IT LIKE AN ENGINEER

Name the boundary, expected state transition, evidence produced and one failure mode before you run or modify anything.

DE10-07 - Practice, validation and mastery

GUIDED REBUILD

Inspect POSTGRES_PASSWORD_FILE and per-service secret grant. Reproduce the matching course asset once and save concrete evidence.

CHECKPOINT / VALIDATE

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence.

RECOVER

If a secret appears in Dockerfile, Git or logs, rotate it and redesign injection.

INDEPENDENT ATTEMPT

Close this page and the external visual. Rebuild the competency from the provided requirement, not by copying the example. Save first attempt even when wrong.

FRESH TRANSFER

Change at least one identifier/configuration/data interval/failure condition and prove the same principle still holds.

EXPLAIN BACK

In 60-90 seconds explain what you built, why the control exists, how you validated it, and what failure would invalidate your claim.

MASTER

Master only when you can reproduce the changed task without the lesson/review open and defend one trade-off.

DE10-08 - Why orchestration exists

OUTCOME

Explain why orchestration exists above scripts/cron.

EXPLAIN

Orchestration models dependencies, schedules, state, retries, observability and reruns. It coordinates work; it does not make unsafe tasks idempotent.

WHY IT MATTERS

A pipeline that works once is not enough. The operational contract must still be reproducible, observable and safe when configuration, time, failures or retries change.

ASSIGNED VISUAL / REFERENCE

Astronomer - Intro to Apache Airflow: From basics to agentic workflows

Current orchestration mental model.

Official: Airflow 3.3.1 overview

WORKING MODEL / DEMONSTRATION

Map M09 pipeline steps into wait -> extract -> validate -> transform -> load -> reconcile.

READ IT LIKE AN ENGINEER

Name the boundary, expected state transition, evidence produced and one failure mode before you run or modify anything.

DE10-08 - Practice, validation and mastery

GUIDED REBUILD

Map M09 pipeline steps into wait -> extract -> validate -> transform -> load -> reconcile. Reproduce the matching course asset once and save concrete evidence.

CHECKPOINT / VALIDATE

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence.

RECOVER

If retrying the DAG duplicates data, repair task/data semantics before increasing retries.

INDEPENDENT ATTEMPT

Close this page and the external visual. Rebuild the competency from the provided requirement, not by copying the example. Save first attempt even when wrong.

FRESH TRANSFER

Change at least one identifier/configuration/data interval/failure condition and prove the same principle still holds.

EXPLAIN BACK

In 60-90 seconds explain what you built, why the control exists, how you validated it, and what failure would invalidate your claim.

MASTER

Master only when you can reproduce the changed task without the lesson/review open and defend one trade-off.

DE10-09 - Airflow architecture and UI

OUTCOME

Identify scheduler, DAG processing, executor/workers, metadata/API/UI roles at a practical level.

EXPLAIN

The scheduler decides runnable work; executors/workers run tasks; metadata tracks state; UI/API expose control/inspection. Airflow 3 uses a stable Task SDK/public interface.

WHY IT MATTERS

A pipeline that works once is not enough. The operational contract must still be reproducible, observable and safe when configuration, time, failures or retries change.

ASSIGNED VISUAL / REFERENCE

[Astronomer - An introduction to the Airflow UI](#)

Current UI reference.

[Official: Airflow 3.3.1 overview](#)

WORKING MODEL / DEMONSTRATION

Use the current UI reference to locate DAG runs, task logs and task states.

READ IT LIKE AN ENGINEER

Name the boundary, expected state transition, evidence produced and one failure mode before you run or modify anything.

DE10-09 - Practice, validation and mastery

GUIDED REBUILD

Use the current UI reference to locate DAG runs, task logs and task states. Reproduce the matching course asset once and save concrete evidence.

CHECKPOINT / VALIDATE

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence.

RECOVER

If UI says failed, read the specific task/run/log rather than restarting everything.

INDEPENDENT ATTEMPT

Close this page and the external visual. Rebuild the competency from the provided requirement, not by copying the example. Save first attempt even when wrong.

FRESH TRANSFER

Change at least one identifier/configuration/data interval/failure condition and prove the same principle still holds.

EXPLAIN BACK

In 60-90 seconds explain what you built, why the control exists, how you validated it, and what failure would invalidate your claim.

MASTER

Master only when you can reproduce the changed task without the lesson/review open and defend one trade-off.

DE10-10 - DAGs, tasks and dependencies

OUTCOME

Define a DAG as tasks plus dependency rules, not as a giant script.

EXPLAIN

Tasks should have clear inputs/outputs, bounded side effects and explicit dependencies. Independent tasks can run concurrently when safe.

WHY IT MATTERS

A pipeline that works once is not enough. The operational contract must still be reproducible, observable and safe when configuration, time, failures or retries change.

ASSIGNED VISUAL / REFERENCE

[Astronomer - Intro to Apache Airflow: From basics to agentic workflows](#)

DAGs/tasks/dependencies.

[Official: Airflow Task SDK examples](#)

WORKING MODEL / DEMONSTRATION

Read m10_daily_orders.py dependency chain.

READ IT LIKE AN ENGINEER

Name the boundary, expected state transition, evidence produced and one failure mode before you run or modify anything.

DE10-10 - Practice, validation and mastery

GUIDED REBUILD

Read m10_daily_orders.py dependency chain. Reproduce the matching course asset once and save concrete evidence.

CHECKPOINT / VALIDATE

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence.

RECOVER

If ordering is only hidden in code, make dependency structure explicit.

INDEPENDENT ATTEMPT

Close this page and the external visual. Rebuild the competency from the provided requirement, not by copying the example. Save first attempt even when wrong.

FRESH TRANSFER

Change at least one identifier/configuration/data interval/failure condition and prove the same principle still holds.

EXPLAIN BACK

In 60-90 seconds explain what you built, why the control exists, how you validated it, and what failure would invalidate your claim.

MASTER

Master only when you can reproduce the changed task without the lesson/review open and defend one trade-off.

DE10-11 - TaskFlow, operators and task communication

OUTCOME

Use TaskFlow for Python task composition and keep task communication small.

EXPLAIN

TaskFlow automatically wires returned values through Airflow task communication. XCom-style payloads are control/small metadata, not a replacement for data storage.

WHY IT MATTERS

A pipeline that works once is not enough. The operational contract must still be reproducible, observable and safe when configuration, time, failures or retries change.

ASSIGNED VISUAL / REFERENCE

[Astronomer - Intro to Apache Airflow: From basics to agentic workflows](#)

TaskFlow/decorators/operators.

[Official: Airflow Task SDK](#)

WORKING MODEL / DEMONSTRATION

Extract returns rows, load returns a summary, reconcile consumes summary.

READ IT LIKE AN ENGINEER

Name the boundary, expected state transition, evidence produced and one failure mode before you run or modify anything.

DE10-11 - Practice, validation and mastery

GUIDED REBUILD

Extract returns rows, load returns a summary, reconcile consumes summary. Reproduce the matching course asset once and save concrete evidence.

CHECKPOINT / VALIDATE

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence.

RECOVER

If huge data is passed between tasks, persist it externally and pass a reference.

INDEPENDENT ATTEMPT

Close this page and the external visual. Rebuild the competency from the provided requirement, not by copying the example. Save first attempt even when wrong.

FRESH TRANSFER

Change at least one identifier/configuration/data interval/failure condition and prove the same principle still holds.

EXPLAIN BACK

In 60-90 seconds explain what you built, why the control exists, how you validated it, and what failure would invalidate your claim.

MASTER

Master only when you can reproduce the changed task without the lesson/review open and defend one trade-off.

DE10-12 - Scheduling and data intervals

OUTCOME

Reason about schedule, logical date/data interval, start date and manual runs.

EXPLAIN

A scheduled run represents a data interval, not simply the wall-clock instant the task starts. Use UTC-aware reasoning and explicit interval-based data queries.

WHY IT MATTERS

A pipeline that works once is not enough. The operational contract must still be reproducible, observable and safe when configuration, time, failures or retries change.

ASSIGNED VISUAL / REFERENCE

[Astronomer - Schedule DAGs in Apache Airflow](#)

Schedules, logical/data intervals, backfill distinction.

[Official: Airflow scheduler](#)

WORKING MODEL / DEMONSTRATION

For a daily DAG, state which interval a run should process.

READ IT LIKE AN ENGINEER

Name the boundary, expected state transition, evidence produced and one failure mode before you run or modify anything.

DE10-12 - Practice, validation and mastery

GUIDED REBUILD

For a daily DAG, state which interval a run should process. Reproduce the matching course asset once and save concrete evidence.

CHECKPOINT / VALIDATE

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence.

RECOVER

If a run reads now() blindly, backfills may process the wrong slice.

INDEPENDENT ATTEMPT

Close this page and the external visual. Rebuild the competency from the provided requirement, not by copying the example. Save first attempt even when wrong.

FRESH TRANSFER

Change at least one identifier/configuration/data interval/failure condition and prove the same principle still holds.

EXPLAIN BACK

In 60-90 seconds explain what you built, why the control exists, how you validated it, and what failure would invalidate your claim.

MASTER

Master only when you can reproduce the changed task without the lesson/review open and defend one trade-off.

DE10-13 - Retries, timeouts and failure behavior

OUTCOME

Configure retries for transient failures and timeouts for bounded execution.

EXPLAIN

Retry only plausibly transient work. A retry is safe only when side effects are idempotent/transactional. `execution_timeout` bounds a task attempt.

WHY IT MATTERS

A pipeline that works once is not enough. The operational contract must still be reproducible, observable and safe when configuration, time, failures or retries change.

ASSIGNED VISUAL / REFERENCE

[Astronomer - Best practices for debugging your Airflow DAGs](#)

Failure diagnosis and retries.

[Official: Airflow 3.3.1 task/DAG concepts](#)

WORKING MODEL / DEMONSTRATION

Simulator extract fails twice then succeeds on attempt 3; validation failure does not retry blindly.

READ IT LIKE AN ENGINEER

Name the boundary, expected state transition, evidence produced and one failure mode before you run or modify anything.

DE10-13 - Practice, validation and mastery

GUIDED REBUILD

Simulator extract fails twice then succeeds on attempt 3; validation failure does not retry blindly. Reproduce the matching course asset once and save concrete evidence.

CHECKPOINT / VALIDATE

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence.

RECOVER

If retries amplify damage, stop and repair idempotency or classify permanent errors.

INDEPENDENT ATTEMPT

Close this page and the external visual. Rebuild the competency from the provided requirement, not by copying the example. Save first attempt even when wrong.

FRESH TRANSFER

Change at least one identifier/configuration/data interval/failure condition and prove the same principle still holds.

EXPLAIN BACK

In 60-90 seconds explain what you built, why the control exists, how you validated it, and what failure would invalidate your claim.

MASTER

Master only when you can reproduce the changed task without the lesson/review open and defend one trade-off.

DE10-14 - Parameters and task communication

OUTCOME

Use runtime Params for validated run configuration and task communication for outputs.

EXPLAIN

Params are configuration supplied/defaulted for a run and JSON-Schema validated. Task outputs carry produced metadata/results. Do not confuse them.

WHY IT MATTERS

A pipeline that works once is not enough. The operational contract must still be reproducible, observable and safe when configuration, time, failures or retries change.

ASSIGNED VISUAL / REFERENCE

Astronomer - Create and use params in Airflow

Runtime Params and precedence.

Official: Airflow Params

WORKING MODEL / DEMONSTRATION

min_amount is a validated DAG Param; task results form the simulator xcom dictionary.

READ IT LIKE AN ENGINEER

Name the boundary, expected state transition, evidence produced and one failure mode before you run or modify anything.

DE10-14 - Practice, validation and mastery

GUIDED REBUILD

min_amount is a validated DAG Param; task results form the simulator xcom dictionary. Reproduce the matching course asset once and save concrete evidence.

CHECKPOINT / VALIDATE

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence.

RECOVER

If a parameter changes business scope, record it in run metadata.

INDEPENDENT ATTEMPT

Close this page and the external visual. Rebuild the competency from the provided requirement, not by copying the example. Save first attempt even when wrong.

FRESH TRANSFER

Change at least one identifier/configuration/data interval/failure condition and prove the same principle still holds.

EXPLAIN BACK

In 60-90 seconds explain what you built, why the control exists, how you validated it, and what failure would invalidate your claim.

MASTER

Master only when you can reproduce the changed task without the lesson/review open and defend one trade-off.

DE10-15 - Connections, variables and secrets

OUTCOME

Keep connection credentials and sensitive Variables out of DAG source.

EXPLAIN

Connections identify external systems; secrets can come from environment/metastore/external backends. Masking helps but is not permission control.

WHY IT MATTERS

A pipeline that works once is not enough. The operational contract must still be reproducible, observable and safe when configuration, time, failures or retries change.

ASSIGNED VISUAL / REFERENCE

[Astronomer - Manage connections in Apache Airflow](#)

Connections and secret-safe configuration.

[Official: Airflow Connections](#)

[Official: Airflow secrets](#)

WORKING MODEL / DEMONSTRATION

Explain why conn_id belongs in DAG code but password does not.

READ IT LIKE AN ENGINEER

Name the boundary, expected state transition, evidence produced and one failure mode before you run or modify anything.

DE10-15 - Practice, validation and mastery

GUIDED REBUILD

Explain why conn_id belongs in DAG code but password does not. Reproduce the matching course asset once and save concrete evidence.

CHECKPOINT / VALIDATE

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence.

RECOVER

If plaintext credentials are committed, treat it as exposure and rotate.

INDEPENDENT ATTEMPT

Close this page and the external visual. Rebuild the competency from the provided requirement, not by copying the example. Save first attempt even when wrong.

FRESH TRANSFER

Change at least one identifier/configuration/data interval/failure condition and prove the same principle still holds.

EXPLAIN BACK

In 60-90 seconds explain what you built, why the control exists, how you validated it, and what failure would invalidate your claim.

MASTER

Master only when you can reproduce the changed task without the lesson/review open and defend one trade-off.

DE10-16 - Sensors and waiting

OUTCOME

Wait for external conditions without wasting worker capacity indefinitely.

EXPLAIN

Sensors poll/wait. Reschedule or deferrable patterns can release worker resources; every wait needs a timeout/failure policy.

WHY IT MATTERS

A pipeline that works once is not enough. The operational contract must still be reproducible, observable and safe when configuration, time, failures or retries change.

ASSIGNED VISUAL / REFERENCE

[Astronomer - Airflow sensors](#)

Sensor/waiting patterns.

[Official: Airflow deferrable operators and triggers](#)

WORKING MODEL / DEMONSTRATION

Simulator wait_input has bounded polls and fails downstream safely.

READ IT LIKE AN ENGINEER

Name the boundary, expected state transition, evidence produced and one failure mode before you run or modify anything.

DE10-16 - Practice, validation and mastery

GUIDED REBUILD

Simulator wait_input has bounded polls and fails downstream safely. Reproduce the matching course asset once and save concrete evidence.

CHECKPOINT / VALIDATE

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence.

RECOVER

If a sensor waits forever, add timeout and decide skip/fail/escalate behavior.

INDEPENDENT ATTEMPT

Close this page and the external visual. Rebuild the competency from the provided requirement, not by copying the example. Save first attempt even when wrong.

FRESH TRANSFER

Change at least one identifier/configuration/data interval/failure condition and prove the same principle still holds.

EXPLAIN BACK

In 60-90 seconds explain what you built, why the control exists, how you validated it, and what failure would invalidate your claim.

MASTER

Master only when you can reproduce the changed task without the lesson/review open and defend one trade-off.

DE10-17 - Logs and debugging

OUTCOME

Debug from task state and logs using a repeatable ladder.

EXPLAIN

Start with failing task -> attempt -> exception -> input/config -> upstream state -> environment. Logs should include run/task identifiers but never secrets.

WHY IT MATTERS

A pipeline that works once is not enough. The operational contract must still be reproducible, observable and safe when configuration, time, failures or retries change.

ASSIGNED VISUAL / REFERENCE

[Astronomer - Best practices for debugging your Airflow DAGs](#)

Logs/debugging/observability.

[Official: Airflow logging for tasks](#)

WORKING MODEL / DEMONSTRATION

Read simulator logs for transient retries and validation failure.

READ IT LIKE AN ENGINEER

Name the boundary, expected state transition, evidence produced and one failure mode before you run or modify anything.

DE10-17 - Practice, validation and mastery

GUIDED REBUILD

Read simulator logs for transient retries and validation failure. Reproduce the matching course asset once and save concrete evidence.

CHECKPOINT / VALIDATE

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence.

RECOVER

If you rerun without identifying cause, you may hide an intermittent defect.

INDEPENDENT ATTEMPT

Close this page and the external visual. Rebuild the competency from the provided requirement, not by copying the example. Save first attempt even when wrong.

FRESH TRANSFER

Change at least one identifier/configuration/data interval/failure condition and prove the same principle still holds.

EXPLAIN BACK

In 60-90 seconds explain what you built, why the control exists, how you validated it, and what failure would invalidate your claim.

MASTER

Master only when you can reproduce the changed task without the lesson/review open and defend one trade-off.

DE10-18 - Catchup, backfills and reprocessing

OUTCOME

Backfill historical intervals without changing business meaning or duplicating outputs.

EXPLAIN

Backfills recreate runs for past data intervals. Tasks must use interval inputs and idempotent writes. Catchup is a scheduling choice; backfill is an explicit operation.

WHY IT MATTERS

A pipeline that works once is not enough. The operational contract must still be reproducible, observable and safe when configuration, time, failures or retries change.

ASSIGNED VISUAL / REFERENCE

[Astronomer - Building ETL/ELT pipelines with Apache Airflow 3](#)

Backfills, mapping and data-quality gates.

[Official: Airflow CLI reference](#)

WORKING MODEL / DEMONSTRATION

Simulator backfills three named dates and produces independent summaries.

READ IT LIKE AN ENGINEER

Name the boundary, expected state transition, evidence produced and one failure mode before you run or modify anything.

DE10-18 - Practice, validation and mastery

GUIDED REBUILD

Simulator backfills three named dates and produces independent summaries. Reproduce the matching course asset once and save concrete evidence.

CHECKPOINT / VALIDATE

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence.

RECOVER

If historical rerun uses current date/config accidentally, stop and make interval/config explicit.

INDEPENDENT ATTEMPT

Close this page and the external visual. Rebuild the competency from the provided requirement, not by copying the example. Save first attempt even when wrong.

FRESH TRANSFER

Change at least one identifier/configuration/data interval/failure condition and prove the same principle still holds.

EXPLAIN BACK

In 60-90 seconds explain what you built, why the control exists, how you validated it, and what failure would invalidate your claim.

MASTER

Master only when you can reproduce the changed task without the lesson/review open and defend one trade-off.

DE10-19 - Integration and production-style orchestration

OUTCOME

Defend an end-to-end orchestration design including deployment, state, recovery and observability.

EXPLAIN

Production readiness combines reproducible runtime, clear dependencies, safe secrets, bounded retries/waits, idempotent data operations, reconciliation, logs and tested reprocessing.

WHY IT MATTERS

A pipeline that works once is not enough. The operational contract must still be reproducible, observable and safe when configuration, time, failures or retries change.

ASSIGNED VISUAL / REFERENCE

[Astronomer - Best Practices for Running Apache Airflow in Production](#)

Production scaling, operations and debugging.

[Official: Airflow scheduler](#)

[Official: Docker Compose production](#)

WORKING MODEL / DEMONSTRATION

Combine Docker static lab with Airflow DAG + simulator evidence; identify what still requires real runtime.

READ IT LIKE AN ENGINEER

Name the boundary, expected state transition, evidence produced and one failure mode before you run or modify anything.

DE10-19 - Practice, validation and mastery

GUIDED REBUILD

Combine Docker static lab with Airflow DAG + simulator evidence; identify what still requires real runtime. Reproduce the matching course asset once and save concrete evidence.

CHECKPOINT / VALIDATE

Prove the expected state/output using an independent check. For Docker lessons validate file/config structure; for Airflow lessons validate DAG/task/dependency/run semantics and evidence.

RECOVER

If a diagram cannot explain failure/recovery and evidence, it is not an operational design.

INDEPENDENT ATTEMPT

Close this page and the external visual. Rebuild the competency from the provided requirement, not by copying the example. Save first attempt even when wrong.

FRESH TRANSFER

Change at least one identifier/configuration/data interval/failure condition and prove the same principle still holds.

EXPLAIN BACK

In 60-90 seconds explain what you built, why the control exists, how you validated it, and what failure would invalidate your claim.

MASTER

Master only when you can reproduce the changed task without the lesson/review open and defend one trade-off.